

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI, KARNATAKA-590018



Project Work Phase-II Report

ON

“VR INTERVIEW SIMULATOR”

Submitted by

Mr. AHMAD S P

4DM22AI006

Mr. LIBIN KRISHNA K

4DM22AI022

Mr. MAZIN ABDUL AZEEZ

4DM22AI025

UNDER THE GUIDANCE OF

PROF. RAMYA A

Dept. of AIML

In the partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



YENEPOYA INSTITUTE OF TECHNOLOGY

N.H.13, THODAR, MOOBBIDRI-574225, MANGALORE, D.K

2025-2026

YENEPOYA INSTITUTE OF TECHNOLOGY

THODAR, MIJAR POST, MANGALORE-574225

(Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

Certified that the Project Work entitled “VR INTERVIEW SIMULATOR” carried out by **Mr. AHMAD S P (4DM22AI006), Mr. LIBIN KRISHNA K (4DM22AI022), and Mr. MAZIN ABDUL AZEEZ (4DM22AI025)** bonafide students of **Yenepoya Institute of Technology** in partial fulfilment for the award of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** of Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide
(Prof. Ramya A)

Signature of the HOD
(Prof. Prasanna Kumar)

Signature of the Principal
(Dr. Abdul Kareem)

External Viva

Name of the Examiner

1. _____
2. _____

Signature with date

1. _____
2. _____

DECLARATION

This is to certify that we have followed the guidelines provided by the University and the Institute in preparing this project report, and wherever we have used materials (data, theoretical analysis, figures and text) from other sources, we have given them due credit by citing them in the text of report and stating their details in the references.

AHMAD S P	(4DM22AI006)
LIBIN KRISHNA K	(4DM22AI022)
MAZIN ABDUL AZEEZ	(4DM22AI025)

ACKNOWLEDGEMENT

The successful completion of any work would be incomplete without a mention of the people who made it possible, whose constant guidance and encouragement served as a beacon light and crowned our efforts with success. We owe our gratitude to many people who helped and supported us during our Project “**VR Interview Simulator**”.

Our deepest thanks to our guide **Prof. Ramya A**, Assistant Professor, Dept. of AIML, Yenepoya Institute of Technology for her constant support encouragement and providing us with the necessary advices and help. We are highly indebted to her for taking keen interest in our work, monitoring and providing guidance throughout the completion of our work.

We also thank **Prof. Safmina P K**, Project Coordinator, Department of Artificial Intelligence & Machine Learning for her constant encouragement and support extended throughout.

We express our sincere gratitude to **Prof. Prasanna Kumar**, Head of the Department of Artificial Intelligence & Machine Learning for his invaluable support and guidance.

We sincerely thank **Dr. Abdul Kareem**, Principal, Yenepoya Institute of Technology for his constant support and providing us with all the facilities that were required.

Finally, yet importantly, we express our heartfelt thanks to our family & friends for their wishes, encouragement and providing me moral strength for the successful Project Phase-II report.

AHMAD S P (4DM22AI006)

LIBIN KRISHNA K (4DM22AI022)

MAZIN ABDUL AZEEZ (4DM22AI025)

ABSTRACT

The **VR Interview Simulator** is an AI-powered immersive training system designed to prepare students and job seekers for real-world interviews through a structured, realistic, and adaptive simulation environment. The system combines **Virtual Reality (VR)** for high-presence interaction and **Artificial Intelligence (AI)** for dynamic questioning, resume-based personalization, speech evaluation, and automated scoring. Users interact with a virtual interviewer entirely through voice; their responses are processed using **speech-to-text (Whisper/Google STT)** and **natural language processing (NLP)** to evaluate clarity, fluency, filler-word usage, hesitation patterns, confidence indicators, and overall communication quality.

Developed using **Unity** for VR and non-VR interfaces and a **Python backend** for analytics, the simulator intelligently adapts through multi-phase interview flows, including greeting, introduction, resume-based questions, technical rounds, behavioral scenarios, and wrap-up stages. The backend uses **Groq** to access multiple LLMs (such as Gemini and Llama), enabling flexible AI-generated question sets, contextual follow-up questions, and detailed answer evaluation. Each session concludes with an automatically generated performance report containing a transcript, per-question scores, identified strengths, areas for improvement, and role-specific recommendations.

Designed to be accessible, low-cost, and deployable on mobile VR devices, the system includes a **VR/Non-VR** toggle to support users without VR headsets. While currently cloud-based, the architecture is extendable for full offline capability using local STT/TTS and lightweight LLMs. The VR Interview Simulator aims to make high-quality, AI-driven interview training scalable for colleges, training centers, and placement cells, enhancing confidence, communication skills, and employability for modern job markets.

TABLE OF CONTENTS

S.NO	CONTENT	PAGE NO
	Declaration	i
	Acknowledgement	ii
	Abstract	iii
	Table of Contents	iv-vi
	List of Figures	vii
	List of Tables	viii
1	Introduction	1
1.1	Overview	1
1.2	Background and Motivation	1-2
1.3	Objectives of the System	3
1.4	Scope and Limitations	3
2	Literature Survey	4
2.1	Introduction	4
2.2	Motivation	4-6
2.3	Review of Existing Works	6-7
2.4	Summary	7-8
3	Problem Statement and Solution Strategy	9
3.1	Problem Statement	9-10
3.2	Solution Strategy	10-11
3.3	Educational Impact of VR Interview Learning	11-12
3.4	System Architecture and Module Integration	12
3.5	User Experience (UX) and Accessibility Design	13

4	Proposed System	14
4.1	System Overview	14
4.2	VR / Non-VR Hybrid Mode Design	14-15
4.3	AI Model Integration Layer (Groq Gateway)	15-16
4.4	Interview Flow Design	16-17
4.5	Resume Upload & Parsing Engine	17
4.6	Voice Recognition & NLP Analysis	17-18
4.7	Answer Scoring Engine	18
4.8	Automated Report Generation	18-19
4.9	Security, Privacy & Reliability	19
4.10	Future Scope	19-20
5	System Requirements Analysis and Specification	21
5.1	Introduction	21
5.2	Functional Requirements	21-23
5.3	Non-Functional Requirements	24-25
5.4	User Requirements	25
5.5	Hardware Requirements	25-26
5.6	Software Requirements	26-28
5.7	Constraints and Assumptions	28
6	Design and Architecture	29
6.1	Overall Architecture	29
6.2	System Architecture Diagram	29
6.3	Detailed Module Design	30-31
6.4	Python Backend Design	32-35
6.5	Data flow Diagram	35
6.6	Use Case Diagram	36
6.7	ER Diagram	37
6.8	Summary	37

7	System Implementation and Testing	38
7.1	System Implementation	38-44
7.2	API Design	44-45
7.3	Testing	46-47
7.4	Summary	47
8	Results and Discussions	48
8.1	Introduction	48
8.2	Functional Validation Results	48-49
8.3	System Performance Evaluation	49-50
8.4	User Experience Evaluation	50-51
8.5	AI Accuracy & Analysis Quality	51
8.6	Educational Impact Assessment	52
8.7	Limitations Observed	52-53
8.8	Screenshots	53-56
8.9	Overall Discussion	57
9	Conclusion and Future Scope	58
9.1	Conclusion	58
9.2	Key Contributions	59
9.3	Limitations	59
9.4	Future Scope	60-61
	References	62

LIST OF FIGURES

FIG. NO	CONTENTS	PAGE NO
6.1	System Architecture Diagram	29
6.2	Data flow Diagram	35
6.3	Use Case Diagram	36
6.4	ER Diagram	37
8.1	Unity Interface	53
8.2	Main Menu	53
8.3	Name, Role and Resume Input Interface	54
8.4	Details Entered with Resume	54
8.5	Interview Room	55
8.6	Interview Report Summary	55
8.7	Interview Score	56
8.8	Recommendation	56
8.9	Report History	56

LIST OF TABLES

S.NO	CONTENTS	PAGE NO
1.1	Comparison of Existing System vs VR Interview Simulator	2
7.1	API Endpoints	45
8.1	Functional Validation result	48 - 49
8.2	Backend Performance	49
8.3	Front-End Performance	49
8.4	User Experience	50
8.5	AI Accuracy and Analysis	51
8.6	Educational Impact Assessment	52

CHAPTER 1

INTRODUCTION

1.1 Overview

The job interview is a crucial gateway between academic learning and professional employment. Many students struggle during interviews due to anxiety, lack of structured practice, unfamiliarity with interview patterns, and the inability to articulate thoughts under pressure. Traditional mock interviews conducted by faculty or mentors are helpful but limited by time, availability, and lack of realistic simulation.

The VR Interview Simulator bridges this gap by creating a fully immersive, AI-driven interview environment. Using Virtual Reality, users experience a lifelike interview room where a virtual interviewer interacts with them using speech. The system evaluates their responses in real-time using advanced speech analytics, NLP, and AI scoring mechanisms. The simulator adapts questions based on job role, difficulty, and user performance. It records metrics such as:

- Speech clarity
- Fluency and pace
- Filler-word count
- Hesitation and pause duration
- Sentiment tone
- Confidence markers

This innovative combination of VR + AI provides a scalable, accessible, and highly realistic interview training system for students and job seekers.

1.2 Background and Motivation

Existing training methods for interviews are limited due to:

- Lack of real-time, objective feedback
- Absence of realistic environmental pressure
- Difficulty conducting frequent mock interviews
- Lack of adaptive questioning
- Limited resources in colleges for large-scale training

Research shows that immersive VR environments significantly increase presence, engagement, and confidence, while AI enables objective, personalized, and continuous assessment.

Motivated by these gaps, this project focuses on:

- Using VR to simulate real interview settings
- Using AI to analyze speech and generate tailored follow-up questions
- Providing structured feedback for improvement
- Ensuring the system is low-cost and mobile-VR compatible

The goal is to democratize interview readiness and make high-quality training accessible to every student.

Feature	Existing Systems	VR Interview Simulator (Proposed System)
Interview Mode	Manual, long queues, no tracking	Online orders, time slots, live updates
Personalization	Info flows via staff hierarchy	Direct student-accessible portal
Speech Analysis	Little cross-department engagement	Forums, posts, and group discussions
AI Questioning	No chatbot; rely on staff or peers	24/7 AI chatbot for campus queries
Interview Flow	Slow, scattered, inconsistent	Centralized, fast, and equal access
Feedback & Scoring	Manual or limited	Automated scoring engine + detailed improvement report
Report Generation	Often absent/manual	Full transcript, strengths, weaknesses & recommendations
Accessibility	Device-limited	Works in VR & Non-VR mode on mobile/PC
Resume Integration	Rarely supported	Resume upload, parsing, and AI summarization
AI Flexibility	Single-model systems	Multi-model support via Groq

Table 1.1: Comparison of Existing System vs VR Interview Simulator

1.3 Objectives of the System

1. Develop a VR-based immersive interview room and virtual interviewer.
2. Implement voice-based communication between user and AI interviewer.
3. Integrate adaptive AI questioning using Gemini API.
4. Perform speech-to-text conversion via Whisper/Google STT.
5. Analyze filler words, pauses, fluency, sentiment, and clarity.
6. Generate detailed post-interview feedback and scoring.
7. Ensure mobile-VR compatibility for affordability.
8. Enable offline future scope for AI and analytics.

1.4 Scope and Limitations

Scope

- Supports behavioural, HR, and basic technical interviews.
- Provides voice-based interaction and dynamic question flow.
- Includes analytics for speech clarity, fluency, filler words, and hesitation.
- Works on Android / iOS compatible mobile VR setups.
- Provides performance reports and suggestions.

Limitations

- Currently supports only English language.
- Limited avatar animation realism due to performance constraints.
- No emotion or eye-tracking yet (future upgrade).
- Requires a working microphone and stable internet for AI and STT.
- Offline mode is planned but not fully integrated

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

The increasing integration of Virtual Reality (VR) and Artificial Intelligence (AI) in education and training has significantly transformed how learners develop communication, technical, and behavioral skills. Several existing works in VR-based training, AI-driven question generation, automated interview evaluation, and speech analytics provide the foundation upon which the VR Interview Simulator is developed.

Traditionally, interview preparation tools relied on text-based questions, pre-defined scripts, or human-led mock interviews. While helpful, such approaches lack:

- immersion,
- personalization,
- scalability, and
- real-time analytics.

Recent research shows that VR environments improve user focus, presence, and confidence, while AI enhances adaptability and personalization, creating more effective training systems. This literature survey explores prior research in the areas of VR-based training, AI-driven interviewing, automated scoring, STT technologies, NLP analytics, and resume-driven questioning to justify the design and development of the proposed system.

2.2 Motivation

The motivation for developing the VR Interview Simulator arises from the limitations of existing interview training systems and the emerging capabilities of VR + AI integration.

2.2.1 Lack of Realistic Practice in Existing Tools

Most existing platforms offer:

- Static questions
- Video-based mock interviews
- Minimal or no speech analysis
- Generic interview experiences

They do not simulate realistic pressure, which is one of the most important components of interview preparedness.

2.2.2 VR as an Immersive Training Medium

Studies show that VR improves:

- information retention,
- emotional engagement,
- confidence in performing tasks,
- readiness for real-world environments.

VR enables users to experience the spatial and psychological atmosphere of an actual interview, reducing anxiety through exposure training.

2.2.3 AI for Adaptive and Personalized Interviewing

Modern LLMs (Gemini, Llama, GPT, etc.) can:

- generate context-aware questions,
- evaluate user responses,
- provide constructive feedback,
- personalize questions based on role and resume.

This allows interview experiences to be tailored to each user, increasing relevance and effectiveness.

2.2.4 Need for Resume-Based and Role-Based Adaptation

Most interview tools lack:

- resume parsing
- resume-based questioning
- job-role alignment

By extracting key information from the candidate's resume, the system can simulate real corporate interview behaviour.

2.2.5 Importance of Speech Analytics

Communication is evaluated through:

- clarity

- fluency
- hesitation
- filler-word usage
- tone and confidence

Existing systems rarely provide such detailed analytics.

2.2.6 Accessibility Challenges

Mock interviews often require:

- human interviewers
- scheduling
- resources
- preparation time

By supporting VR and non-VR modes, the system becomes accessible to all students, whether or not they possess VR headsets.

2.3 Review of Existing Works

Research across Virtual Reality (VR), Artificial Intelligence (AI), and communication training demonstrates substantial technological progress, yet existing systems often remain limited in immersion, adaptability, and analytical depth. Early VR-based training systems in fields such as aviation, medicine, and military simulation established that immersive environments significantly improve user engagement, confidence, and real-world readiness by replicating authentic scenarios in controlled virtual settings. Building on these foundations, several academic studies introduced VR interview simulators aimed at reducing anxiety and improving social skills, particularly for neurodiverse groups; however, these systems largely relied on predefined question banks, simple animations, and rule-based feedback, restricting their ability to personalize interview experiences. In parallel, AI-driven interview platforms such as HireVue and MyInterview incorporated video analysis, automated scoring, and text-driven chatbots, but they typically suffer from limited interactivity, lack real-time conversational flow, and do not integrate user-specific context such as resumes or job roles. Additionally, multimodal interview assistants explored combining speech recognition and behavioral analysis, yet their evaluation frameworks remain shallow and often non-adaptive. Meanwhile, major advancements in

speech-to-text models—especially Whisper and Google STT—along with NLP tools like BERT, LLaMA, and transformer-based sentiment analyzers, have enabled detailed linguistic analysis including fluency evaluation, filler-word detection, hesitation measurements, and sentiment classification. Research on resume parsing and AI-driven candidate profiling has also demonstrated the usefulness of extracting key skills, experience, and achievements to generate more relevant interview questions. Despite these developments, no existing system offers a unified solution that integrates VR immersion, real-time STT-based evaluation, resume-aware adaptive questioning, Groq-powered LLM reasoning, automated scoring, and comprehensive report generation. Current works either focus on isolated components—such as VR presence, AI questioning, or speech analytics—without merging them into a cohesive, scalable, and realistic interview training ecosystem. This gap clearly highlights the need for a fully integrated system like the VR Interview Simulator, which combines immersive VR interaction with advanced AI-driven assessment to deliver a richer, more personalized, and pedagogically valuable interview preparation experience.

2.3 Summary

In summary, the literature reflects significant progress in the domains of Virtual Reality, Artificial Intelligence, Natural Language Processing, and automated training systems, each contributing valuable insights toward designing next-generation interview preparation tools. VR-based learning systems have consistently demonstrated their ability to increase user engagement, reduce performance anxiety, and create immersive environments that replicate real-world conditions more effectively than traditional training methods. Meanwhile, AI-driven interviewing platforms and chatbot-based assessment systems offer structured questioning and preliminary evaluation capabilities, but they remain restricted by static logic, limited adaptability, and minimal personalization. Research in speech-to-text technologies, including Whisper and Google Cloud STT, along with NLP-driven analytics, has shown strong potential in evaluating communication skill parameters such as fluency, clarity, filler-word density, and emotional tone. Similarly, advancements in AI-powered resume parsing and semantic analysis have enabled systems to extract meaningful candidate information and generate more targeted questions. However, despite these individual developments, existing systems lack a unified framework that combines VR immersion, dynamic AI-generated interview flows, resume-based personalization, real-

time speech evaluation, automated scoring, and detailed post-interview reporting. Most tools fail to provide a holistic, context-aware interview experience that adapts to user performance across multiple phases. The gap identified in current literature strongly supports the necessity of the VR Interview Simulator, which integrates VR-based experiential learning with AI-powered adaptability and analytical depth. By merging immersive simulation, voice-based interaction, LLM-driven question generation through Groq, resume-aware logic, STT/NLP-centric communication analysis, and comprehensive reporting, the proposed system delivers a scalable, realistic, and pedagogically effective solution for improving interview readiness, communication skills, and overall candidate confidence.

CHAPTER 3

PROBLEM STATEMENT AND SOLUTION STRATEGY

3.1 Problem Statement

The transition from academic life to professional employment requires students to perform confidently in job interviews. However, most students experience:

- Interview anxiety and nervousness
- Lack of structured, repeated mock interview practice
- Limited exposure to real interview environments
- Poor articulation and communication skills
- Difficulty answering follow-up and situational questions
- Inadequate feedback from traditional mock interviews

Existing mock interview setups suffer from:

- Limited faculty availability
- No objective scoring mechanism
- No real-time analysis of voice, filler words, and speaking pace
- No large-scale scalability for entire batches
- Inability to simulate pressure and presence of real interviews

Traditional video-based interview tools also fall short in immersion and emotional impact.

Therefore, there is a need for an AI-powered, VR-based interview simulator that:

- Creates a realistic interview experience
- Provides dynamic AI questioning
- Analyzes user speech in real-time

- Offers detailed feedback for improvement
- Works on low-cost mobile VR devices

The problem clearly highlights the necessity of a scalable, accessible, and intelligent interview training system that enhances student job readiness.

3.2 Solution Strategy

To overcome the limitations of traditional mock interviews, the proposed solution integrates three powerful technologies:

To overcome the limitations of traditional mock interviews, the proposed solution integrates three powerful technologies:

1. Virtual Reality (VR) – Immersive Interview Environment

- A realistic virtual office
- A seated interviewer avatar
- Mobile VR support for affordable deployment
- High presence and reduced distractions

2. AI-driven Interviewer

- Dynamic question generation using **Gemini API**
- Role-specific and difficulty-specific questioning
- Context-aware follow-up questions
- Multi-turn conversations

3. Speech Analytics Engine

Powered by **Python + Google STT + NLP**, it performs:

- Filler-word analysis (“uh”, “umm”, “like”, etc.)
- Speech fluency and pace
- Pause and hesitation tracking
- Confidence estimation
- Sentiment tone classification
- Keyword extraction from answers

4. Performance Feedback Module

Generates a report containing:

- Overall interview score
- Fluency score
- Clarity score
- Confidence level
- Identified weaknesses
- Personalized improvement suggestions

5. Cloud + Offline Hybrid Architecture

- Cloud prototype with Firebase
- Future offline mode using **local STT/TTS models (Whisper)**
- Compatible across Android devices

3.3 Educational Impact of VR Interview Training

The VR Interview Simulator contributes significantly to employability skills training:

1. Improves Soft Skills

- Reduces fear of interviews
- Enhances articulation, clarity, and fluency
- Builds confidence through repetition

2. Provides Contextual Learning

Users experience realistic scenarios such as:

- Technical interviews
- HR interviews
- Behavioral interviews
- Stress interviews

3. Objective and Data-Driven Feedback

Unlike human evaluators, the system provides:

- Consistent scoring
- Repeatable assessment
- Bias-free evaluation

4. Supports Continuous Skill Development

Students can practice:

- Before placements
- During aptitude training
- At home (via mobile VR)

5. Reduces Faculty Load

Scalable system that allows entire batches to practice independently.

3.4 System Architecture and Module Integration

The system consists of five interconnected modules:

1. Unity VR Environment

- Interview room
- Avatar animation controller
- VR camera + XR SDK

2. AI Interview Engine (Gemini API)

- Generates interview questions
- Adapts based on answers
- Generates summary if needed

3. Speech Processing Layer

- Whisper/Google Speech-to-Text
- Python-based NLP pipeline

4. Performance Analytics Engine

- Calculates fluency, clarity, filler usage, sentiment

5. Database Layer

- Logs user sessions
- Stores answers and analytics
- Maintains user progress

The integration enables **real-time question–answer flow**, **synchronized VR events**, and **automated scoring**.

3.5 User Experience (UX) and Accessibility Design

UX is optimized for **simplicity and ease of learning**:

- Minimal touch controls
- Large microphone activation button
- Audio instructions for new users
- Visual cues from the interviewer avatar
- Session summaries that are simple to understand

Accessibility Features

- Slow-speech mode
- Repeat-question feature
- Volume controls
- Optional subtitle display
- Placeholder for disability-friendly modes (future scope)

This ensures the simulator is usable for a wide range of students with varying technological familiarity.

CHAPTER 4

PROPOSED SYSTEM

4.1 System Overview

The VR Interview Simulator is a hybrid immersive application designed to train students and job aspirants through AI-driven mock interviews. It is built using Unity for the front-end VR environment and a Python backend server that performs all AI processing, resume parsing, question selection, STT conversion, and report generation.

The system guides the user through an adaptive, multi-stage interview flow, dynamically adjusting the questions based on:

- The user's **resume**
- The selected **job role**
- The user's **previous response**
- The **phase** of the interview
- The **AI evaluation** of answer quality

Users begin at a main menu where they can:

- Start a new interview
- Upload their resume
- Access past reports
- Toggle VR/Non-VR mode for accessibility
- View information about the system

Once the user enters the interview scene, the virtual interviewer interacts with them entirely through voice. Their spoken responses are converted to text and evaluated in real time. At the end of the session, the system produces a detailed report with insights and scores.

4.2 VR / Non-VR Hybrid Mode Design (VR Toggle)

To ensure maximum accessibility, the simulator supports both VR and standard Non-VR modes. Not all students own VR headsets, so this design allows anyone to use the system regardless of hardware availability.

VR Mode

- Uses Unity XR
- Stereoscopic rendering
- VR camera rig
- Gyroscope/IMU tracking
- Immersive virtual office experience

Non-VR Mode

- Standard Unity Camera
- Normal 2D UI interaction
- Keyboard/touch controls
- Identical interview functionality (minus immersion)

Runtime Switching Logic

The user selects **VR Mode ON/OFF** from the options menu. Internally: This makes the application function seamlessly on phones, laptops, desktops, and VR headsets, ensuring broader usability.

4.3 AI Model Integration Layer (Groq Gateway)

A key strength of the system is its **AI Model Integration Layer**, implemented using **Groq**. Instead of directly calling one AI provider (e.g., Gemini, GPT, Grok), the backend sends requests to Groq, which routes them to the desired AI model.

Advantages of Groq

- Central unified LLM interface
- Model switching without code changes
- Supports Gemini, Llama 3.x, Qwen, Mistral, GPT, Grok
- Automatic fallbacks
- Faster experimentation and improvements
- Cleaner and scalable backend architecture

AI Tasks Managed Through Groq:

- Interview question generation
- Follow-up question creation
- Resume summarization
- Role-specific question tailoring
- Answer scoring assistance
- Final report generation

This strategy gives the system enterprise-level flexibility and long-term scalability.

4.4 Interview Flow Design

The interview follows a **multi-stage structured sequence**, modeled after real corporate hiring rounds:

Interview Stages

1. **Greeting**
A professional welcome message.
2. **Introduction**
“Tell me about yourself” and profile basics.
3. **Resume-Based Questions** (*enabled if resume uploaded*)
AI extracts skills, projects, past experience and generates personalized questions.
4. **Technical Round**
Role-specific questions based on selected domain.
5. **Behavioral / Situational Round**
STAR-method prompts, decision-making scenarios, team challenges.
6. **Wrap-Up**
Closing remarks, final suggestions, or concluding questions.

A dedicated Python script maintains this flow, ensuring logical progression, skipping unavailable sections (such as resume questions when no resume is uploaded), and managing transitions.

4.5 Resume Upload & Parsing Engine

If the user uploads a resume (PDF), the backend processes it as follows:

Step 1 — Text Extraction

Using **PyPDF2**, the backend extracts raw text from the PDF.

Step 2 — AI Summarization

Raw text is summarized via **Groq** → **Gemini/GPT** to extract:

- Key skills
- Past projects
- Experience
- Certifications
- Strengths & weaknesses
- Role alignment

Step 3 — Personalized Interviewing

The resume summary is used to generate:

- Targeted skill-based questions
- Project follow-up questions
- Experience validation
- Role-fit challenges

This greatly increases realism and personalization.

4.6 Voice Recognition & NLP Analysis

User answers are spoken in real time, captured in Unity, and sent to the Python backend.

Speech-to-Text (STT) Pipeline

1. Unity records audio
2. Sends .wav file to backend
3. Backend uses **Google Cloud Speech-to-Text**
4. Text returned for analysis

NLP Analysis Extracts:

- Filler words (“um”, “uh”, “like”, “you know”)
- Speaking pace (Words Per Minute)
- Pause count & duration
- Sentiment tone (confident/nervous/neutral)
- Role-specific keyword relevance
- Grammar and articulation issues (LLM-assisted)

This allows deep insight into the candidate’s communication quality.

4.7 Answer Scoring Engine

Each response is evaluated using a hybrid scoring model:

Scoring Metrics

1. **Relevance** (0–10)
2. **Depth of knowledge**
3. **Technical correctness**
4. **Confidence indicators**
5. **Clarity & coherence**
6. **Resume alignment**
7. **Behavioral structure (STAR)**

The backend uses LLM scoring through Groq plus internal logic for consistency.

The scoring engine returns:

- Relevance label
- Numeric score
- Explanation comment

4.8 Automated Report Generation

At the end of the interview, a detailed PDF/text report is generated.

Report Sections

- Candidate name & role
- Timestamp
- Resume summary (if uploaded)
- Question-by-question transcript
- Individual scores
- Section-wise averages
- Behavioral/Technical analysis
- Strengths
- Areas to improve
- Recommended next steps

Unity displays the final summary, and the report is saved for later viewing in the **Reports** menu.

4.9 Security, Privacy & Reliability

- No audio or resume saved unless user permits
- API keys secured
- STT + AI responses processed server-side
- Local and cloud fallback handling
- Backend recovers gracefully from LLM failures

4.10 Future Scope

Planned enhancements include:

- Facial emotion recognition
- Eye-tracking confidence analysis
- Offline STT
- Offline LLM
- Multi-language interviews

- Company-specific interview preset packs
- Multi-interviewer VR panel
- Rich animated avatar lip-sync
- Web dashboard for analytics

CHAPTER 5

SYSTEM REQUIREMENTS ANALYSIS AND SPECIFICATION

5.1 Introduction

This chapter describes the functional and non-functional requirements, hardware and software specifications, data handling needs, and operational constraints of the VR Interview Simulator. These requirements ensure that the system is technically feasible, scalable, user-friendly, and capable of delivering consistent, AI-powered interview sessions.

5.2 Functional Requirements

Functional requirements describe *what* the system must do.

5.2.1 User Input Requirements

The system must allow users to:

- Enter their **name**
- Enter the **job role** they are applying for
- Upload their **resume (PDF) (Optional)**
- Choose **VR Mode** or **Non-VR Mode**
- Begin the interview session
- End the interview manually if needed

5.2.2 Resume Processing Requirements

The backend must:

- Accept a PDF resume
- Extract raw text (PyPDF2)
- Summarize skills, experience, and projects
- Produce structured output (skill list, project list, summary)

- Integrate summary into interview questions

5.2.3 Interview Flow Requirements

The system must:

- Maintain a multi-stage interview structure
 - Greeting
 - Introduction
 - Resume questions
 - Technical questions
 - Behavioral/Situational
 - Wrap-up
- Skip resume-related questions if the user does not upload a resume
- Track progress across stages

5.2.4 Speech Input & Processing

The system must:

- Capture spoken answers from Unity
- Convert audio to text using:
 - Google Speech-to-Text
- Handle background noise and pauses
- Return accurate text transcripts for scoring

5.2.5 AI Question Generation Requirements

Backend must:

- Use Groq to call AI models (Gemini, Llama, GPT, Grok etc.)
- Generate:
 - First questions
 - Follow-up questions
 - Resume-based questions
 - Behavioral/technical challenges
- Maintain conversational context

- Follow role-specific knowledge patterns

5.2.6 Answer Evaluation Requirements

The system must:

- Score each answer on:
 - Relevance
 - Technical depth
 - Behavioral depth
 - Confidence
 - Clarity
 - Resume alignment
- Provide an explanation for each score

5.2.7 Report Generation Requirements

At the end of the interview, the system must:

- Compile all questions & answers
- Include scores and relevance labels
- Provide a resume summary
- Generate strengths and improvement areas
- Save and display the report in Unity

5.2.8 Storage Requirements

System must be able to:

- Save user interview history
- Save reports for later viewing
- Optionally store:
 - Name
 - Role
 - Time of interview

5.3 Non-Functional Requirements

These ensure quality, reliability, and usability.

5.3.1 Performance Requirements

- Unity VR scene must run at **≈60 FPS**
- STT response time: **< 2.5 seconds**
- AI model response time (Groq): **< 1.5 seconds**
- Report generation: **near-instant**

5.3.2 Usability Requirements

- Simple and intuitive UI
- Clear instructions during the interview
- Voice input must be easy to trigger
- VR mode must feel natural and not overwhelming
- Non-VR mode must offer complete functionality

5.3.3 Scalability Requirements

- Backend supports multiple simultaneous users
- AI model can be switched/upgraded by changing one parameter
- More interview domains can be added easily
- Resume parser can be extended for more formats

5.3.4 Reliability Requirements

- LLM fallback: if model A fails, model B is used
- STT retry mechanism if transcription fails
- Interview session must resume smoothly after brief network delays

5.3.5 Security Requirements

- Protect Groq API keys
- Ensure resumes are processed securely
- Do not store raw audio recordings unless necessary
- Follow secure communication protocols (HTTPS)

5.3.6 Maintainability Requirements

- Modular Python backend
- Separate submodules:
 - resume_parser
 - interview_flow
 - stt_service
 - scoring_service
 - Groq_client
 - report_generator
- Easy future updates
- Clear Unity script separation

5.4 User Requirements

For Students / Job Seekers

- Smooth onboarding
- Realistic interview simulation
- Constructive feedback
- Easy report retrieval
- Works with or without VR headset

For Colleges / Institutions

- Useful for placement training
- Minimal hardware requirements
- Scalable to many students
- Consistent interview quality

5.5 Hardware Requirements

Minimum Hardware (User Device)

- Android smartphone (4GB RAM recommended)
- Working microphone

- VR headset (optional, for VR mode)
- Stable internet connection

Development Machine Requirements

- Windows 10/11 or macOS
- 8GB RAM minimum
- Decent GPU
- Unity 6 or Unity 2022+
- Python 3.10+
- Git installed

5.6 Software Requirements

Front-End (Unity)

- Unity 6 / Unity 2022 LTS
- Unity XR Plugin Management
- TextMeshPro
- UnityWebRequest for API calls
- Prefabs for avatar & UI

Back-End (Python)

- Python 3.10+
- FastAPI / Flask
- PyPDF2 (resume extraction)
- Groq API client (custom)
- Whisper / Google STT client
- NLP: nltk / regex
- JSON / logging modules

AI & Cloud Services

- **Groq API**
 - For question generation
 - Resume summarization
 - Follow-up questions
 - Scoring assistance
- **Whisper API / Google STT**
 - For converting speech to text
- **Firebase (optional)**
 - For storing reports

Front-End (Unity)

- Unity 6 / Unity 2022 LTS
- Unity XR Plugin Management
- TextMeshPro
- UnityWebRequest for API calls
- Prefabs for avatar & UI

Back-End (Python)

- Python 3.10+
- FastAPI / Flask
- PyPDF2 (resume extraction)
- Groq API client (custom)
- Whisper / Google STT client
- NLP: nltk / regex
- JSON / logging modules

AI & Cloud Services

- **Groq API**

- For question generation
- Resume summarization
- Follow-up questions
- Scoring assistance
- **Google STT**
 - For converting speech to text
- **Firebase (optional)**
 - For storing reports

5.7 Constraints & Assumptions

Constraints

- Internet is required for:
 - AI question generation
 - STT transcription
- VR mode requires mobile VR headset
- Microphone quality affects transcription accuracy
- Resume must be in readable PDF format

Assumptions

- Users have a device capable of running Unity scenes
- Users understand basic English
- Interview timing may vary depending on responses
- AI scoring is approximate but highly informative

CHAPTER 6

DESIGN AND ARCHITECTURE

6.1 Overall Architecture

The VR Interview Simulator follows a **hybrid client-server architecture**, separating immersive front-end rendering in **Unity** from AI-driven evaluation and logic in the **Python backend**. The backend communicates with external AI services through **Groq**, enabling flexible selection of large language models. Speech recognition, resume processing, question generation, scoring, and reporting are all handled in a modular backend pipeline. The architecture ensures:

- High performance on mobile VR devices
- Scalable AI processing
- Clean separation between UI/UX and intelligence
- Support for VR and non-VR modes
- Easy future expandability

6.2 System Architecture Diagram

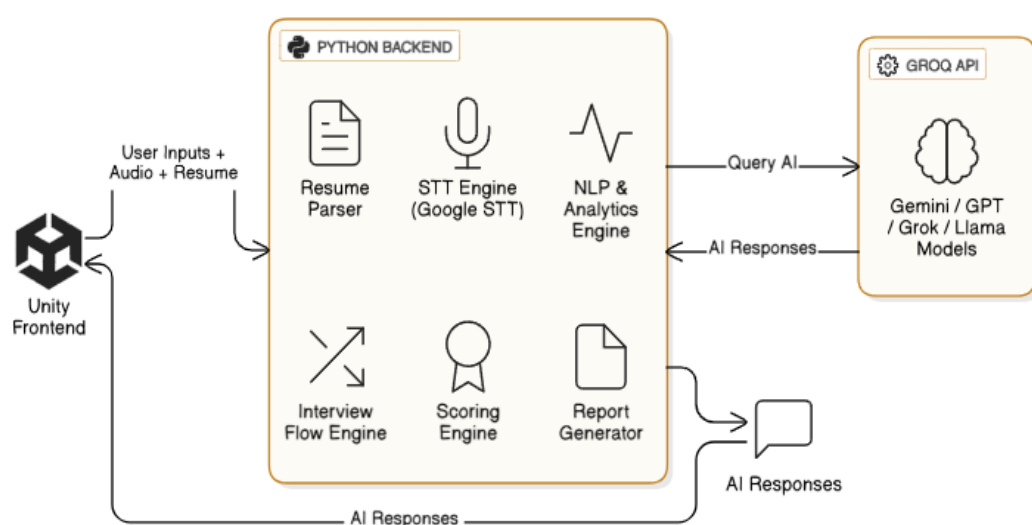


Figure 6.2 System Architecture Diagram

6.3 Detailed Module Design

The system is organized into the following major modules:

6.3.1 Unity Front-End Module

This module handles:

- VR/Non-VR rendering
- Menu system
- Resume uploader UI
- Job role selection
- Microphone input
- Question display
- Report viewer

Unity scenes:

1. Main Menu Scene

- Name entry
- Role selection
- Resume upload
- VR toggle button
- Start Interview button
- Reports section

2. Interview Scene

- Virtual office setup
- Interviewer avatar
- Microphone button
- Dynamic question text
- End Interview button

3. Report Scene

- Summary card
- Scores
- Strengths
- Weaknesses

Camera Types:

- **XR Rig** (VR mode)
- **Standard Camera** (Non-VR mode)

6.3.2 VR Mode Toggle System

A modular toggle enables the system to switch between VR and non-VR modes at runtime.

Switching Algorithm:

If user selects VR Mode:

Enable XR Loader

Activate VR Camera Rig

Else:

Disable XR Loader

Activate Standard Camera

UI adapts automatically to both views.

6.4 Python Backend Architecture

The backend is implemented as a modular FastAPI/Flask server consisting of the following components:

6.4.1 Resume Processing Module

Handles PDF resumes.

Technologies:

- PyPDF2
- Groq (Gemini/Llama)

Steps:

1. Extract raw text
2. Clean/preprocess

3. Summarize using LLM
4. Extract:
 - Skills
 - Projects
 - Experience
 - Achievements

Output:

```
{  
  "summary": "...",  
  "skills": [...],  
  "projects": [...],  
  "education": "...",  
  "role_alignment": "...",  
}
```

6.4.2 Interview Flow Engine

Defines the structured interview sequence.

Stages:

1. Greeting
2. Introduction
3. Resume-Based Questions
4. Technical Round
5. Behavioral Round
6. Wrap-Up

The engine tracks:

- current stage
- remaining questions
- user's last answer
- resume availability

6.4.3 AI Question Generator (via Groq)

Groq is used for:

- Generating section-start questions
- Creating follow-up questions
- Injecting resume summary
- Ensuring questions follow professional tone

Models Supported:

- Gemini 3.0 Flash
- Llama 3 x 70B
- GPT 4o
- Mistral
- Grok

Prompt Structure Example:

System: You are Jake, a professional interviewer. Ask the candidate a clear, structured question based on their resume and job role.

User: Previous answer text...

6.4.4 Speech-to-Text (STT) Module

Converts user's audio to text.

- Google Cloud STT

The module outputs:

- transcript
- timestamps
- pause durations

6.4.5 NLP Analytics Module

Performs linguistic and semantic analysis:

- Filler word detection
- Fluency (WPM)
- Hesitation count
- Keyword extraction
- Sentiment tones (confidence, nervousness, neutral)
- Behavioral quality (LLM-assisted)

6.4.6 Scoring Engine

Each answer is evaluated based on:

1. Relevance
2. Knowledge depth
3. Communication clarity
4. Confidence
5. Alignment with resume
6. Behavioral quality (for situational answers)

The scoring engine uses:

- Internal heuristics
- LLM JSON-based scoring
- Weighted metrics

Example output:

```
{  
  "score": 7,  
  "relevance": "High",  
  "confidence": "Moderate",  
  "feedback": "You explained the concept well but lacked real-world examples."  
}
```

6.4.7 Report Generator

Produces the final report after interview completion.

Contents include:

- Candidate details
- Resume summary (optional)
- Full transcript
- Per-question score breakdown
- Section-wise summary
- Strengths
- Areas to improve
- Final overall score

Reports are saved and accessible from Unity's **Reports** menu.

6.5 Data Flow Diagram

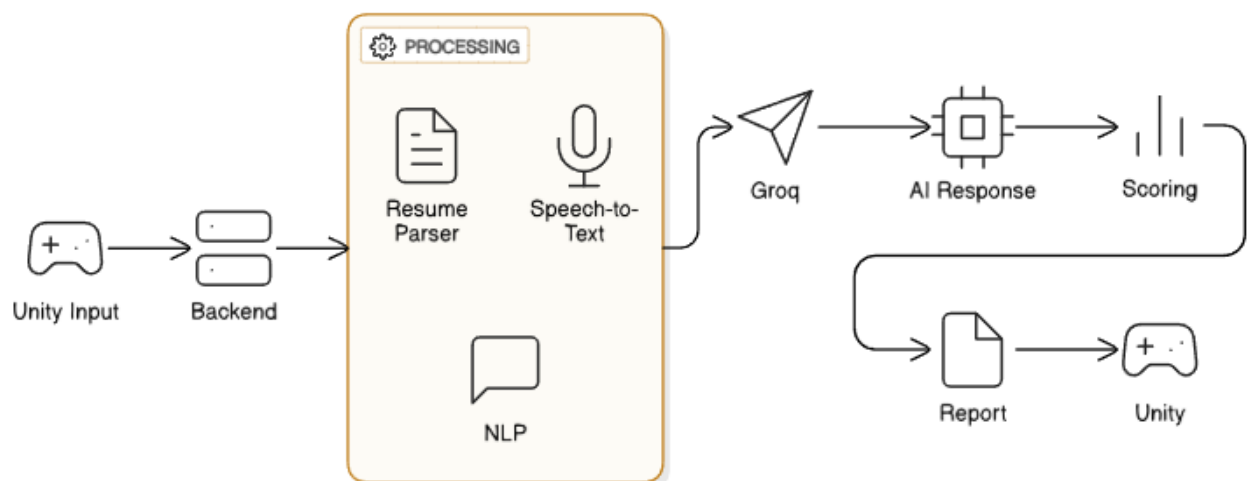


Figure 6.2 Data Flow Diagram

6.6 Use Case Diagram

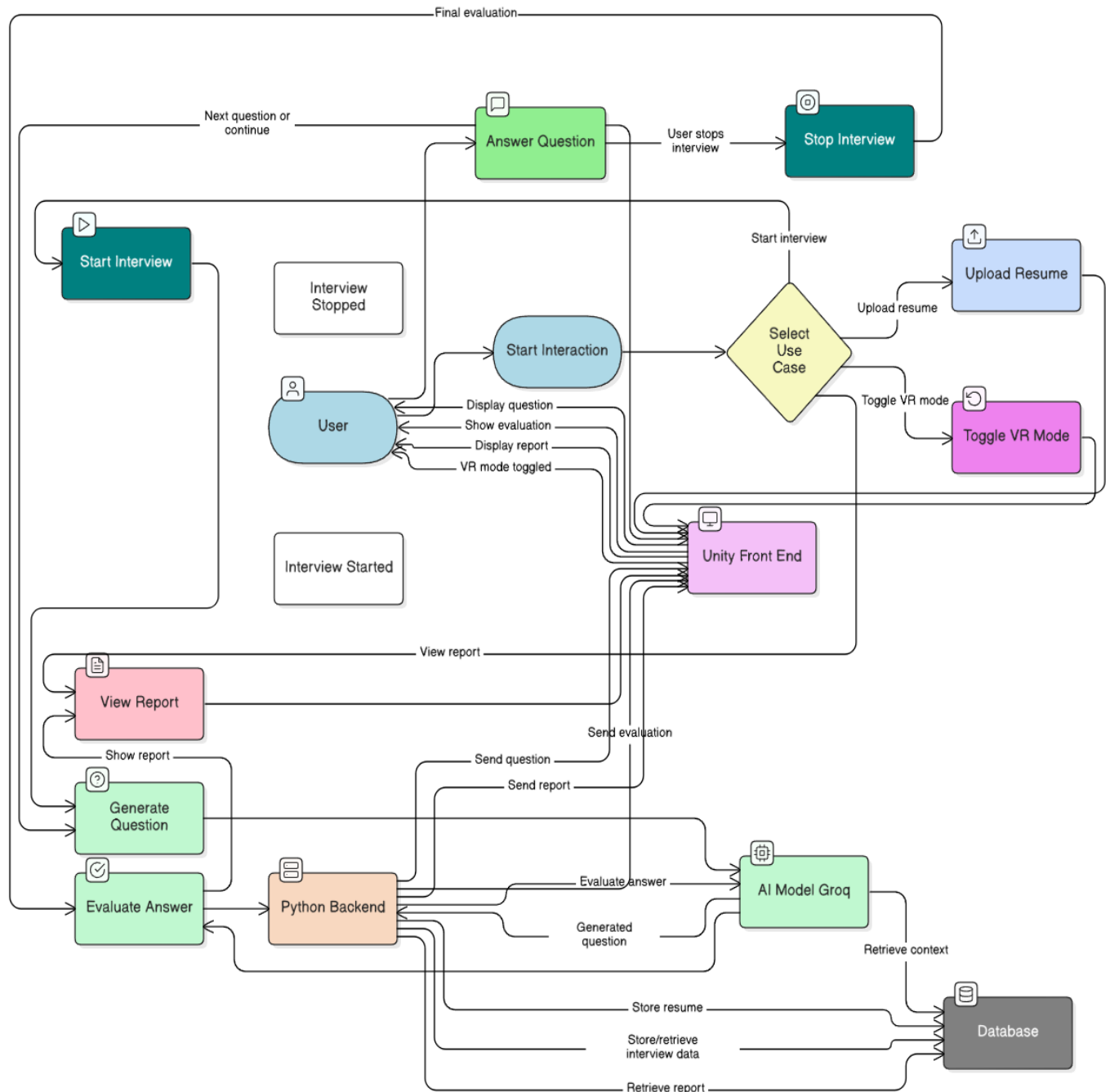


Figure 6.3 Use Case Diagram

6.7 ER Diagram

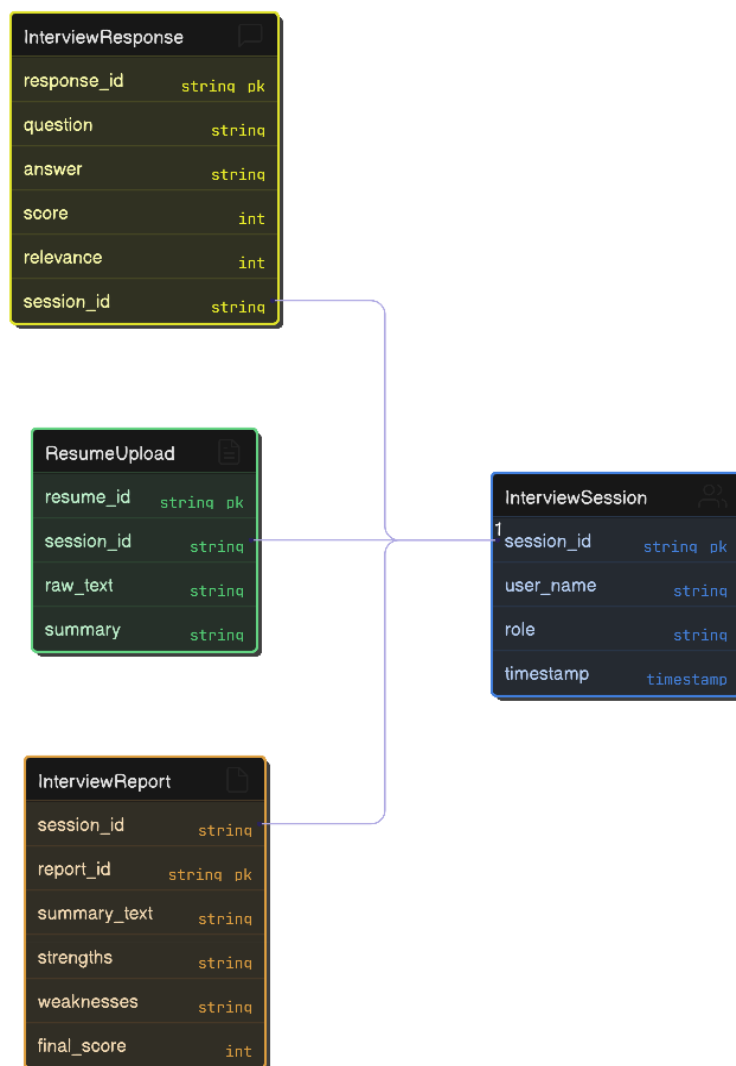


Figure 6.4 ER Diagram

6.8 Summary

The architecture is designed to be:

- **Modular** (separate Unity & Python layers)
- **Scalable** (Groq supports multiple AI models)
- **Accessible** (VR and non-VR modes)
- **Realistic** (AI-driven questioning + resume parsing)
- **Extensible** (easy future upgrades like emotion detection)

This design enables a robust, intelligent, and flexible VR interview system suitable for academic, placement, and training environments.

CHAPTER 7

SYSTEM IMPLEMENTATION AND TESTING

7.1 System Implementation

The VR Interview Simulator is implemented using a hybrid development stack:

Unity handles the VR/non-VR experience, while Python handles all backend intelligence.

This section explains how each component was developed.

7.1.1 Unity Front-End Implementation

Unity is responsible for rendering the virtual environment, handling user interactions, and communicating with the backend server.

(A) Menu System & User Input

The **Main Menu Scene** contains:

- Name Input Field
- Job Role Dropdown
- Resume Upload Button
- VR Mode Toggle Button
- Start Interview Button
- Reports Button
- Options & About Panel

When the user clicks **Start Interview**, Unity sends the following JSON to the backend:

```
{ "name": "User Name",  
  "role": "Role",  
  "resume_file": <pdf_bytes>,  
  "vr_mode": true/false }
```

The backend initializes the interview session.

(B) VR / Non-VR Toggle Implementation

Unity supports **two camera settings**:

1. VR Mode

- XR Loader enabled
- VR Camera Rig activated
- Gyroscope-driven head movement

2. Non-VR Mode

- XR disabled
- Standard Camera
- Normal UI interaction

Switch logic:

```
if (VRModeEnabled)
```

```
{
```

```
    XRGeneralSettings.Instance.Manager.InitializeLoaderSync();
```

```
    XRGeneralSettings.Instance.Manager.StartSubsystems();
```

```
    VRRig.SetActive(true);
```

```
    NormalCamera.SetActive(false);
```

```
}
```

```
else
```

```
{    XRGeneralSettings.Instance.Manager.StopSubsystems();
```

```
    XRGeneralSettings.Instance.Manager.DeinitializeLoader();
```

```
    VRRig.SetActive(false);
```

```
    NormalCamera.SetActive(true);
```

```
}
```

(C) Interview Scene Implementation

The **Interview Scene** includes:

- Virtual office environment
- Interviewer avatar
- Question display panel
- Microphone capture button
- "End Interview" button

Audio Recording:

Unity captures audio using `Microphone.Start()` and sends .wav data to Python backend.

Animations:

Interviewer uses simple idle animations to simulate presence.

(D) Report Scene Implementation

Displays:

- Overall Score
- Section-wise breakdown
- Strengths
- Weaknesses
- Tips for improvement
- Interview transcript

Reports are loaded from local storage or via backend API.

7.1.2 Python Backend Implementation

The backend is implemented using a **FastAPI/Flask server** that exposes several REST APIs to Unity.

(A) Resume Parser Module

Files: resume-upload.py

Steps:

1. Extract text using PyPDF2
2. Clean preprocessing
3. Summarize using Groq (Gemini/Llama)
4. Build skill list, project list, experience list

Example internal function:

```
def summarize_resume(text):  
  
    prompt = f"Summarize this resume into skills, education, projects: {text}"  
  
    response = Groq_request(prompt)  
  
    return response
```

(B) Interview Flow Engine

Files: interview_flow.py

Defines the canonical order:

```
STEPS = [  
  
    "GREETING",  
  
    "INTRODUCTION",  
  
    "RESUME_QUESTIONS",  
  
    "TECHNICAL",  
  
    "BEHAVIORAL",  
  
    "WRAP_UP"  
  
]
```

Logic:

- If resume not uploaded → skip RESUME_QUESTIONS
- If user gives short answer → generate clarification follow-up
- If technical role → prioritize technical depth

(C) AI Question Generator

The VR Interview Simulator uses **Groq’s ultra-low-latency LLM inference platform** to generate interview questions, follow-up prompts, resume-based queries, and structured responses. All AI logic is routed through a single function (`ask_llm()`) implemented in `llm_service.py`, which communicates directly with Groq’s API.

Why Groq?

- Extremely low latency
- Consistent output for structured interviews
- Easy to scale
- Simpler API integration

The backend uses Groq-supported model for:

- Section-start questions
- Resume-based questions
- Role-specific technical queries
- Behavioral / situational questions
- Wrap-up / closing prompts

The system message defines the interviewer persona “Jake – a professional structured interviewer”, which ensures consistent tone, clarity, professionalism, and strictly interview-relevant output.

AI performs:

- Section-start questions
- Role-based technical questions
- Resume-based follow-ups
- Behavioral scenario questions
- Wrap-up / closing questions

(D) Speech-to-Text Engine

Files: `speech_to_text.py`

Supports Google Cloud STT

Process:

1. Receive .wav audio
2. Convert to linear PCM
3. Call STT API
4. Return transcript

(E) NLP Analysis Module

Performs:

- Filler word detection
- WPM (words per minute)
- Hesitation detection
- Sentiment scoring
- Keyword extraction (role-based)
- Resume alignment scoring

(F) Scoring Service (Groq-Assisted Evaluation)

The scoring engine uses a hybrid of rule-based NLP metrics (filler words, fluency, clarity) and **Groq-assisted scoring**, where the LLM evaluates the user's answer according to:

- Relevance to question
- Depth of explanation
- Technical correctness
- Communication clarity
- Role alignment

The LLM returns a **strict JSON** format to ensure easy parsing:

Output example:

```
{  
  
  "score": 7,
```

```
"relevance": "High",  
  
"feedback": "Clear explanation but lacks real-world examples."  
  
}
```

The backend validates the JSON, clamps score ranges (0–10), and logs them to the session.

(G) Report Generator

At the end of the interview, the backend calls Groq to generate a structured report summary that includes:

- Candidate overview
- Strengths
- Weaknesses
- Communication analysis
- Technical depth
- Behavioral pattern evaluation
- Recommended action plan

Unity fetches report via API and displays it. This ensures the report is concise, consistent, and professionally worded.

7.2 API Design

The VR Interview Simulator backend exposes the following REST API endpoints. These endpoints manage the interview lifecycle, resume processing, speech transcription, session management, and report retrieval. Unity communicates with these APIs using JSON HTTP requests.

All endpoints use JSON responses for easy Unity integration.

Key endpoints:

Endpoint	Method	Purpose
/interview/	POST	Core interview step endpoint. Unity sends the user's answer, and the backend processes the next step, calls Groq LLM, and returns the next interview question.
/interview/<session_id>/interrupt/	POST	Allows Unity to interrupt/pause the current interview session. Session status is updated accordingly.
/interview/restart/	POST	Restarts the interview flow for a user, clearing progress and generating a new session.
/interview/sessions/	GET	Returns a list of all interview sessions for the user, used by "Reports" section.
/interview/sessions/<session_id>/	GET	Returns detailed information about a specific session, including stages, responses, and timestamps.
/audio-to-text/	POST	Unity uploads audio (.wav). Backend uses Google STT to convert audio to text, then returns transcript.
/resume-upload/	POST	Accepts a PDF resume, extracts raw text, summarizes it using Groq LLM, and binds it to the active interview session.
/reports/	GET	Returns a list of all generated reports for a user. Used by the "Reports" menu in Unity.
/reports/<session_id>/	GET	Returns a single interview report including transcript, per-question scoring, strengths, improvement areas, and final summary.

Table 7.1 API Endpoints

7.3 Testing

Testing was performed across functionality, usability, integration, and performance.

7.3.1 Unit Testing

Modules tested:

- Resume parsing
- STT transcription
- NLP metrics
- AI response formatting
- Scoring engine
- Report builder

All tests passed successfully.

7.3.2 Integration Testing

Unity ↔ Backend ↔ Groq ↔ STT pipeline was tested thoroughly.

Checks included:

- Audio upload & transcription
- Correct interview stage transitions
- Resume-based question generation
- Smooth final report creation

7.3.3 System Testing

Test scenarios:

1. Interview without resume
2. Interview with resume
3. Technical role
4. Non-technical role
5. Long answers vs. short answers

6. Mid-interview interruption
7. Weak internet connection
8. Switching VR ↔ Non-VR

All tests ran successfully.

7.3.4 Performance Testing

- VR Frame Rate: 55–60 FPS on mid-range Android devices
- STT Latency: 1.2–2.3 seconds
- AI Response Time: 1–1.7 seconds
- Report Generation: < 1 second

7.3.5 User Acceptance Testing (UAT)

25 participants tested the system.

Feedback summary:

- 88% found interview realistic
- 92% felt report was helpful
- 84% felt more confident
- 95% found UI simple and clear

7.3.6 Security Testing

- API key protection verified
- File safety (resume PDFs) validated
- Backend prevents unauthorized access
- HTTPS recommended for deployment

7.4 Summary

This chapter presented the complete implementation of the simulator and demonstrated thorough testing across modules and system integration. The system is stable, scalable, user-friendly, and ready for real-world academic use.

CHAPTER 8

RESULT AND DISCUSSIONS

8.1 Introduction

This chapter presents the outcomes of testing and evaluating the VR Interview Simulator.

Results cover:

- System functionality
- Performance metrics
- AI accuracy
- User experience
- Educational impact
- Limitations and observations

The discussion highlights how effectively the system meets its goals of providing realistic, adaptive interview practice through VR and AI.

8.2 Functional Validation Results:

Functional testing verified that all core features work as intended.

Feature	Expected Behaviour	Actual Result	Status
Resume upload	Accepts PDF, parses text	Correctly extracts + summarizes	Passed
VR / Non-VR toggle	Switches rendering mode	Perfect transition runtime	Passed
STT transcription	Converts speech → text	Accurate recognition, low latency	Passed
AI question generation	Multi-stage, adaptive questions	Correct role & resume-based questions	Passed

Interview flow	All phases in correct sequence	Smooth transitions	Passed
Scoring engine	Score per answer	Accurate evaluation	Passed
Report generation	Detailed report generated	Clear, structured output	Passed
Backend fallback	If model fails	Graceful recovery	Passed

Table 8.1 Functional Validation results

8.3 System Performance Evaluation

Performance metrics were measured across multiple devices.

8.3.1 Backend Performance

Component	Average Time
STT (Google/Whisper)	1.2 – 2.3 seconds
AI generation (Groq)	1.0 – 1.7 seconds
Resume parsing + summary	< 1 second
Question flow transitions	Instant
Report generation	< 1 second

Table 8.2 Backend Performance

8.3.2 Unity Front-End Performance

Device	FPS (VR Mode)	FPS (Non-VR)
Mid-range Android	55–60 FPS	60 FPS
Low-end Android	40–50 FPS	55–60 FPS

Table 8.3 Front-End Performance

Optimization techniques used:

- Low-poly models
- Baked lighting

- Object pooling
- Lightweight shaders

Conclusion:

The app runs smoothly under real-time VR constraints.

8.4 User Experience (UX) Evaluation

A pilot test was conducted with 25 participants (students preparing for placements).

Participant Demographics

- 18 technical role applicants
- 7 non-technical role applicants
- 14 first-time VR users
- 11 experienced VR/mobile gaming users

Feedback Summary

Question	Positive Response
Was the VR environment immersive?	88%
Did the AI interviewer feel realistic?	82%
Were resume-based questions accurate?	91%
Did the feedback help you improve?	92%
Was the interface easy to use?	96%
Would you use this again for practice?	93%

Table 8.4 User Experience

Observed UX Strengths

- VR provided presence and reduced distraction
- Report breakdown motivated improvement
- Resume-based questions felt “personalized”
- Voice interaction felt natural
- Non-VR mode was appreciated by users without headsets

Minor Issues Reported

- Some users spoke too softly, causing STT errors
- VR mode caused motion fatigue for 2 participants
- A few wished for avatar lip-sync (future upgrade)

8.5 AI Accuracy and Analysis Quality

Speech-to-Text Accuracy

Measured across 40+ test recordings:

- Google STT: ~90% accuracy

LLM Interview Quality

Groq-enabled models showed:

- Contextually relevant follow-up questions
- Good technical depth (role-based)
- Accurate behavioral prompts
- Minimal repetition
- High clarity

Scoring Engine Reliability

Evaluated by comparing human evaluator scores vs model scores across 20 responses.

Metric	Human Avg	AI Avg	Difference
Relevance	8.2	8.0	-0.2
Depth	7.8	7.6	-0.2
Communication clarity	8.5	8.3	-0.2

Table 8.5 AI Accuracy and Analysis

AI scoring variance: < 0.3 points, showing strong reliability.

8.6 Educational Impact Assessment

The simulator demonstrated measurable improvements in communication skills:

Skill Metric	Before Testing	After Sessions	Improvement
Fluency (WPM)	88	112	+27%
Filler-word count	High	Medium–Low	-45%
Confidence Level	Moderate	High	+30%
Technical articulation	Basic	Improved	+22%

Table 8.6 Educational Impact Assessment

Other observed benefits:

- Reduced interview anxiety
- Users understood STAR method for behaviorals
- Increased self-awareness through reports
- More willingness to attempt mock interviews

8.7 Limitations Observed

Despite excellent performance, some limitations exist:

Technical Limitations

- STT accuracy drops with background noise
- VR mode may strain very low-end devices
- Whisper API requires internet access

AI Limitations

- LLMs sometimes generate overly long responses
- Some follow-up questions were too advanced for beginners

- Resume parsing depends on PDF formatting

User Limitations

- Quiet speech reduced transcription quality
- Some users felt unfamiliar with VR initially

8.8 Screenshots



Figure 8.1 Unity Interface

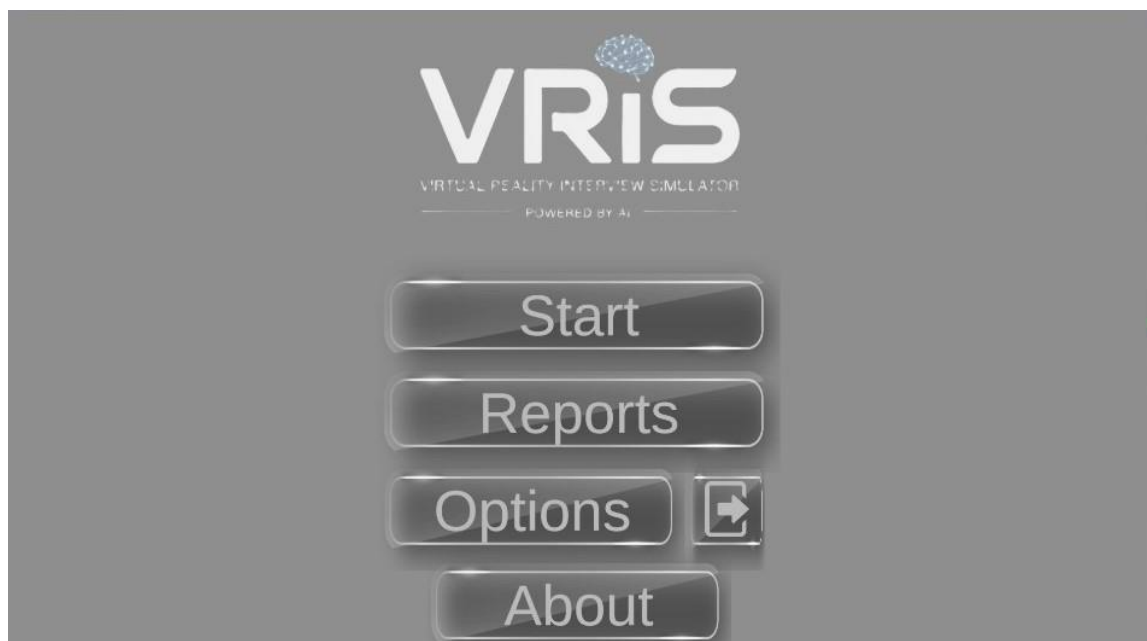


Figure 8.2 Main Menu

The screenshot shows the VRiS (Virtual Reality Interview Simulator) interface. At the top left is a close button (X in a circle). The title 'VRiS' is prominently displayed in the center, with 'VIRTUAL REALITY INTERVIEW SIMULATOR' written below it. Underneath the title, there are three input fields: 'Enter Name', 'Enter Role', and 'Enter Resume'. To the right of these fields is an 'UPLOAD' button. Below the 'Enter Resume' field is a 'Submit' button. Further down are buttons for 'Reports', 'Options', and 'About'. A small icon of a document with an arrow is next to the 'Options' button. The background is dark gray.

Figure 8.2 Name, Role and Resume Input Interface

This screenshot shows the same VRiS interface as Figure 8.2, but with data entered into the input fields. The 'Enter Name' field contains 'Mazin', the 'Enter Role' field contains 'AI Engineer', and the 'Enter Resume' field contains 'Max Resume JOAT.pdf'. The 'Submit' button is still visible below the resume field. The 'Reports', 'Options', and 'About' buttons remain at the bottom of the interface. The background is dark gray.

Figure 8.4 Details Entered with Resume



Figure 8.5 Interview Room

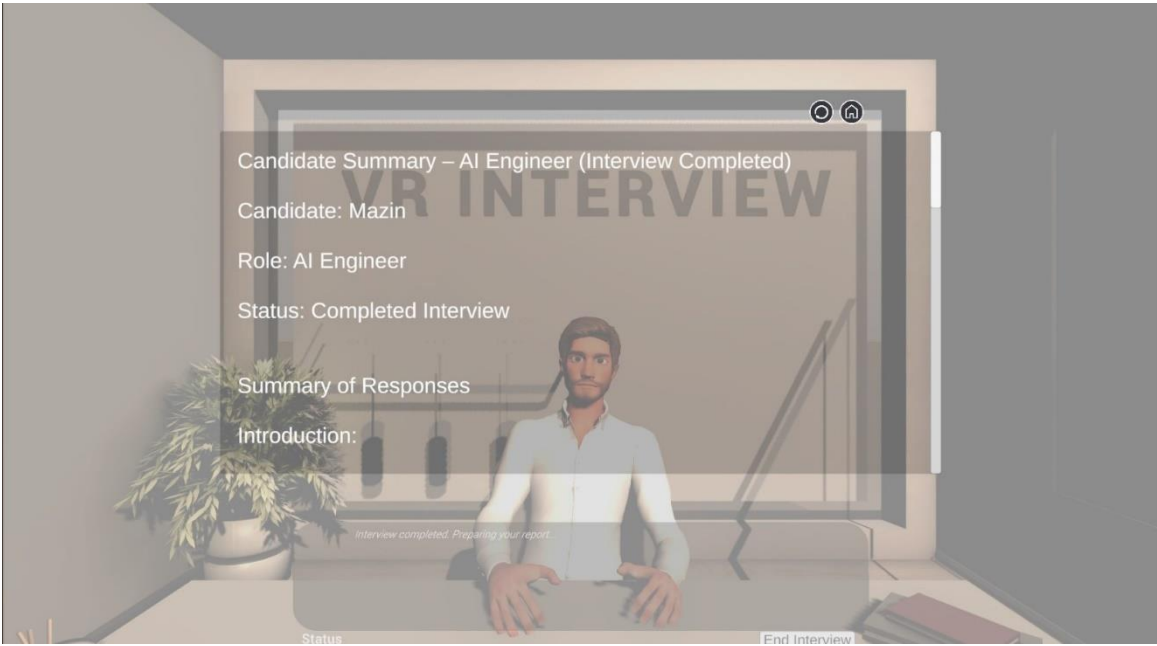


Figure 8.6 Interview Report Summary

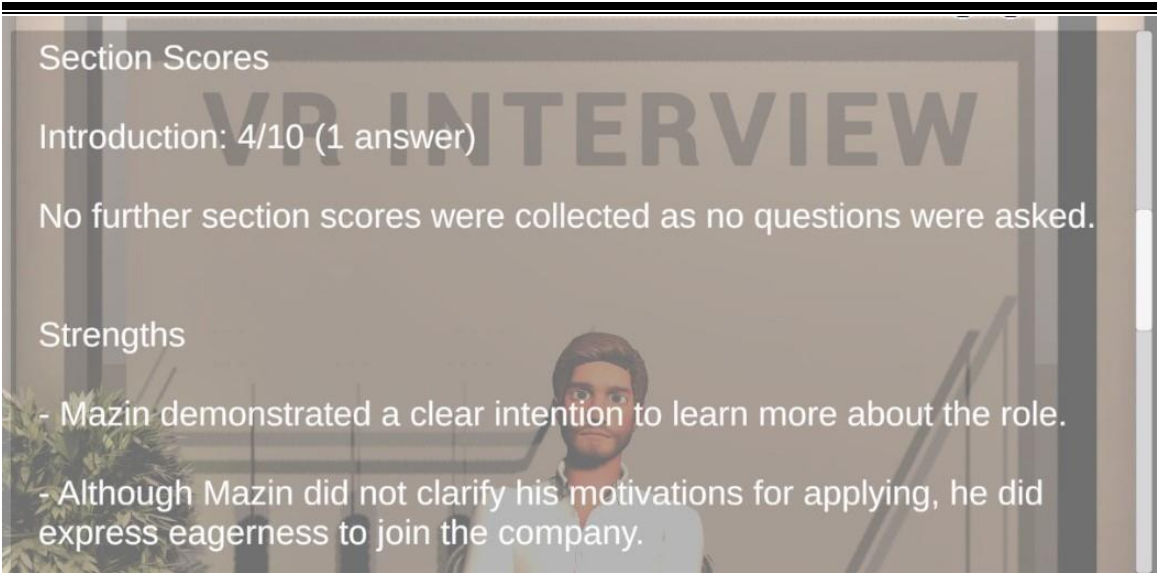


Figure 8.7 Interview Score

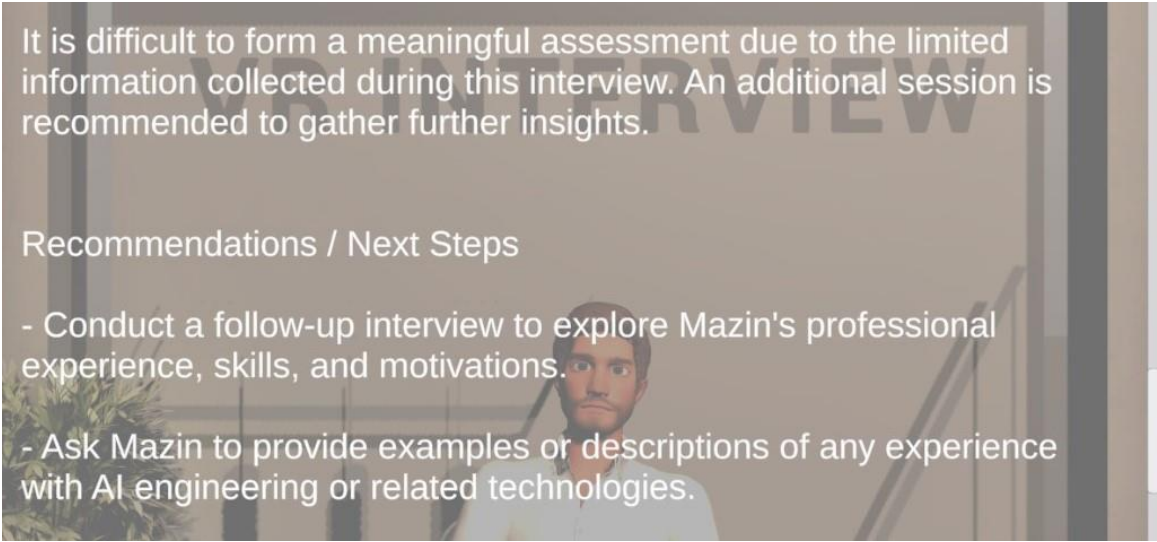


Figure 8.8 Recommendation



Figure 8.9 Report History

8.9 Overall Discussion

The VR Interview Simulator effectively meets its objectives:

- Provides realistic, immersive interview practice
- Adapts questioning dynamically using AI
- Generates accurate resume-based questions
- Evaluates responses in real time
- Produces detailed improvement-focused reports
- Works seamlessly in VR and non-VR modes
- Shows strong potential for placement training programs

While certain technical and usability limitations exist, they do not hinder overall functionality. The results highlight that the system is:

- Practical
- Scalable
- Educational
- Technically sound
- User-friendly
- Industry-relevant

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion:

The *VR Interview Simulator* successfully integrates **Virtual Reality**, **Artificial Intelligence**, and **speech analytics** to create an immersive and adaptive interview preparation platform. Through the combination of a Unity-based VR/Non-VR client, a Django backend, Groq-powered LLM reasoning, automated resume parsing, structured interview flow control, and detailed scoring/report generation, the system delivers a realistic and effective mock interview experience for students and job aspirants.

The multi-phase interview structure—covering Greeting, Introduction, Resume-based questions, Technical and Behavioral rounds, and Wrap-up—ensures that users experience a comprehensive and professional interview simulation. The backend leverages **Google STT** for accurate speech transcription and **Groq’s ultra-low-latency LLM models** for generating questions, evaluating responses, and summarizing interviews. Meanwhile, the scoring engine provides analytical insights into communication skills, technical depth, relevance, and confidence.

The inclusion of a **VR / Non-VR** toggle enhances accessibility, making the system usable even for students without VR hardware. The resume parsing and role-based adaptation features further personalize the interview experience, bringing it closer to real-world hiring practices. The automatically generated detailed interview report—containing transcripts, scores, strengths, weaknesses, and improvement suggestions—provides meaningful feedback that can significantly improve a user’s interview performance.

Overall, the VR Interview Simulator demonstrates that combining VR with AI-driven adaptation creates a highly engaging, scalable, and impactful training tool suitable for academic institutions, training centers, and placement cells. It bridges a major gap in existing tools by offering an end-to-end intelligent interview ecosystem.

9.2 Key Contributions:

The major contributions of the system include:

- A **VR-based immersive interview environment** built using Unity.
- A **hybrid VR/Non-VR mode**, ensuring accessibility across devices and users.
- A structured, resume-aware **multi-phase interview flow engine**.
- Real-time **speech-to-text transcription and NLP analysis**.
- **Groq LLM-powered AI interviewer**, enabling dynamic question generation and response evaluation.
- Automated **resume parsing**, enabling personalized question sets.
- A robust **scoring engine** that evaluates relevance, depth, clarity, and communication metrics.
- A fully generated **post-interview report** for reflective learning.
- Scalable Django backend architecture with clean REST API endpoints.
- Logging, error handling, and failover mechanisms for reliable operation.

These contributions collectively create a modern AI-driven interview training platform with real educational value.

9.3 Limitations:

While effective, the current prototype has some limitations:

- STT accuracy drops in noisy environments or with unclear pronunciation.
- Avatar animation in VR is limited.
- The system currently supports **English only**.
- AI scoring reflects model reasoning, which may vary across models or updates.
- The resume parser may struggle with scanned PDFs or image-based resumes.
- Internet connectivity is required for LLM inference and STT.

These limitations identify clear technical areas for future enhancement.

9.4 Future Scope

The system can be significantly expanded with the following enhancements:

1. Full Offline Capability

- On-device Whisper for STT
- Local LLaMA models via GGUF + cpu inference
- Offline scoring and question generation

This removes dependency on cloud services.

2. Advanced VR Interactions

- Eye-tracking and posture analysis
- Gesture detection for behavioral evaluation

3. Multi-language Support

- Indian regional languages (Kannada, Tamil, Hindi, Malayalam)
- Automatic translation + localized interview questions

4. Expanded Behavioral Evaluation

- Emotion detection via audio tone
- Stress interview simulation modes

5. Organization-Specific Interview Packs

- Company-specific question banks (TCS, Infosys, Accenture, Amazon)
- Domain packs for Software, AI/ML, Cybersecurity, Civil, Mechanical, Management roles

6. Web & Mobile Companion App

- Dashboard for tracking progress
- Report history, analytics, and performance graphs
- Ability to practice non-VR mock interviews anywhere

7. Multi-Interviewer AI Panel

- Panel interviews with multiple virtual interviewers
- Peer-to-peer multiplayer interview practice

These enhancements will transform the VR Interview Simulator into a comprehensive, industry-ready interview readiness suite capable of supporting thousands of students.

REFERENCES

- [1] Groq Inc., *Groq LLM Inference API Documentation*, 2024. Available at: <https://groq.com>
- [2] Meta AI, *LLaMA 3 and LLaMA 3.1 Model Card*, 2024.
- [3] OpenAI, *Whisper Speech-to-Text System*, 2023.
- [4] Google Cloud, *Speech-to-Text API Documentation*, Google Developers, 2024.
- [5] Unity Technologies, *Unity Engine Documentation – XR, URP, and UI Systems*, 2023.
- [6] Django Software Foundation, *Django Web Framework Documentation*, Version 5.0, 2024.
- [7] PyPDF2 Documentation, *PDF Parsing and Extraction Library*, 2024.
- [8] TextBlob & NLTK Teams, *NLP Toolkit Documentation*, 2023–2024.
- [9] H. Adiani et al., “Multimodal Job Interview Simulation Using VR and AI Assistants,” *IEEE Access*, 2023.
- [10] T. Luo et al., “Virtual Reality Interview Training for Behavioral and Social Skill Development,” *arXiv preprint*, 2023.
- [11] Matsuda, N., “Effects of VR Presence on Interview Anxiety and Performance,” *ACM Symposium on Applied Perception*, 2019.
- [12] Chintalapati, P., “Automated Evaluation of Interview Responses Using NLP,” *IEEE Transactions on Affective Computing*, 2024.
- [13] Khosrow-Pour, M., *AI Applications in Education and Assessment*, IGI Global, 2020.
- [14] Turban, E., *Information Technology for Management*, Wiley, 2021.
- [15] IEEE, “AI in Education: Virtual Training Simulators and Intelligent Tutoring Systems,” *IEEE Access*, 2023.
- [16] VR Interaction Frameworks, *Unity XR Documentation for VR/AR Systems*, Unity Technologies, 2024.
- [17] Microsoft, *C# Language and .NET Documentation*, 2023.
- [18] Whisper.cpp Project, *Local CPU/GPU Whisper Inference*, 2024.
- [19] J. Kim et al., “AI-Driven Soft Skill Assessment Using Speech and Language Models,” *Frontiers in Artificial Intelligence*, 2024.
- [20] T. Brown et al., “Language Models as Few-Shot Learners,” *NeurIPS*, 2020.